Cristian Fernandez

A partir de la siguiente definición:

Graph **= Array**(n,LinkedList())

Diseñe una estrategia algorítmica para los siguientes problemas. En cada una de ellas brinde un análisis de las estructuras de datos empleadas y del costo computacional (mejor y peor de los casos)

Ejercicio 1

Dado un grafo G(A,V) no dirigido, determine la cantidad de componentes conexas que posee y el conjunto de nodos perteneciente a cada una de ellas.

Este problema se puede resolver usando **DFS**. DFS (Depth First Search) es un algoritmo que forma un árbol abarcador relacionado a un grafo G(A,V). Cada vértice tiene la propiedad de v.color. Al iniciar el algoritmo el color empieza en blanco. Se comienza por un vértice u cualquiera, se le pone el u.color gris, luego se visita uno de los vértices w con los que comparte arista y que todavía esté blanco, se cambia su w.color a gris, y así sucesivamente hasta que un vértice no esté conectado a un vértice todavía blanco, en ese caso, se retrocede al vértice anterior y a ese último visitado se le cambia el color a negro. Al retroceder, se busca en cada vértice que no hayan quedado vértices conectados de color blanco. En el caso de que se haga el retroceso, no se pueda ir a otro vértice blanco ni retroceder a uno gris, pero todavía queden vértices por recorrer, se sigue por algún otro vértice blanco del grafo el mismo procedimiento.

DFS devuelve un bosque con tantos árboles como componentes conexas tenga el grafo. Así, llevando la cuenta de cada vez que comenzamos por otro vértice no conectado, tendremos la cantidad y estarán compuestas por los vértices de cada árbol.

El costo computacional de DFS utilizando una lista de adyacencia, es de O(|V| + |E|). Esto se debe a que se debe tener en cuenta cada vértice y además chequear cada arista para ver sus conexiones, tanto en su mejor como en su peor caso.

Ejercicio 2

Dado un grafo G(A,V) dirigido y un entero n determinar si dado un par de nodos existe un camino de tamaño n que los conecte

Supongamos que deseamos saber si existe un camino de tamaño n entre un vértice v y un vértice u, y suponiendo que este camino puede repetir vértices y aristas. Se podría usar un **BFS** modificado. BFS recorre el grafo por anchura, dando como resultado un árbol abarcador el cual sirve para encontrar el camino más corto desde un vértice v hasta un vértice u, y colocando en cada vértice la distancia a la que se encuentra del vértice v con el que se inició. Si realizamos una modificación para que el algoritmo no pare cuando encuentre todos los vértices, sino que cada vez que analice un vértice analice todas sus aristas, incluso si está conectada a un vértice ya visitado en otro nivel (se lleva un booleano para determinar que no se haya visitado ya en el nivel actual). Así hasta que la distancia desde el vértice inicial sea igual a n, incluso si todavía no es un árbol abarcador (quedan vértices sin recorrer). Se revisa si alguno de los vértices que tienen distancia igual a n es el vértice u que estábamos buscando, en caso afirmativo se encontró que sí existe un camino de tamaño n entre ese par de vértices.

El costo computacional de esta solución en el peor caso es de O(n(V+E)), esto ocurriría si en cada nivel tuviera que recorrer todas las aristas del grafo. En el mejor caso, cada vértice tendría solo una arista saliente y la complejidad sería de O(n).

Ejercicio 3

Dado un grafo G(A,V) dirigido, determine el conjunto de nodos que pueden alcanzarse desde un nodo u

Se puede resolver utilizando **BFS**, comenzando desde el vértice u. Al finalizar el recorrido, el conjunto de vértices visitados, excluyendo aquellos que no sean alcanzables desde u, constituye la solución. Se utiliza BFS porque permite recorrer todos los vértices a los que se puede llegar siguiendo las aristas en el sentido de su dirección.

Peor caso: O(V+E), el vértice u posee un camino hacia todos los vértices del grafo. Mejor caso: O(V), el nodo u no tiene aristas salientes y el costo es de O(V), debido a la inicialización de los vértices

Ejercicio 4

Dado un grafo G(A,V) dirigido, determine si existe algún ciclo que contenga al nodo u.

Se puede resolver utilizando DFS, comenzando por los vértices adyacentes a u y verificando si alguno de ellos permite alcanzar nuevamente a u. Para ello, se marca inicialmente u como visitado y se utiliza como condición de éxito encontrar una arista que regrese a él durante la exploración. Si esto ocurre, existe un ciclo que contiene a u; de lo contrario, no existe ninguno.

Costo computacional: Peor caso de O(V + E) cuando el algoritmo visita a lo sumo todos los vértices alcanzables desde u una sola vez y examina cada una de sus aristas salientes a lo sumo una vez. Mejor caso de O(1) si el primer vecino es el propio u (arista (u,u)).

Ejercicio 5

Dado un grafo G(A,V) conexo y ponderado, determine un subconjunto de sus aristas que conecte todos los nodos minimizando la suma de los pesos de las aristas utilizadas.

Se puede resolver utilizando el algoritmo de Kruskal o el de Prim, que permiten encontrar un árbol generador de costo mínimo (AACM). Ambos seleccionan aristas de manera que se conecten todos los vértices sin formar ciclos y se minimice el costo total.

Utilizando Kruskal, se ordenan las aristas por peso creciente y se agregan al árbol aquellas que no formen un ciclo con las ya seleccionadas. El algoritmo finaliza cuando se han incorporado V-1 aristas.

Con una implementación basada en conjuntos disjuntos, el costo computacional de Kruskal es de (O(E log E)), debido principalmente al ordenamiento de las aristas.

Ejercicio 6

Dado un grafo G(A,V) no dirigido, determine si al eliminar una determinada arista <u,v> el grafo deja de ser conexo.

Se puede resolver utilizando BFS o DFS desde u, ignorando temporalmente la arista (u,v). Si después del recorrido v no fue visitado, significa que ya no existe otro camino que conecte ambos vértices y, por lo tanto, el grafo deja de ser conexo. En caso contrario, el grafo permanece conexo.

El costo computacional en el peor caso es de O(V+E), ya que puede ser necesario recorrer todo el grafo. Si se encuentra inmediatamente una arista alternativa que conecta u con v, el algoritmo puede finalizar en O(1), suponiendo que las listas de adyacencia permiten consultar directamente las primeras aristas.

Ejercicio 7

Dado un grafo G(A,V) conexo, indique un orden en el cual se puedan eliminar las aristas de manera tal que el grafo que va quedando permanezca conexo hasta eliminar la última arista posible.

Suponiendo que el grafo es no dirigido, se puede resolver utilizando DFS para obtener un árbol generador. Primero se eliminan, en cualquier orden, todas las aristas que no pertenecen al árbol. Esto no desconecta el grafo, ya que las aristas del árbol siguen conectando todos los vértices. Una vez eliminadas dichas aristas, el grafo restante es un árbol, por lo que no se puede eliminar ninguna otra arista sin perder la propiedad de conexo.

El costo computacional es de O(V+E), correspondiente a la construcción del árbol y la identificación de las aristas que no pertenecen a él. El mejor caso es O(V) si el grafo ya es un árbol

Ejercicio 8

Dado un grafo G(A,V) dirigido, determine el conjunto de nodos que pueden alcanzarse desde un nodo u.

Se resuelve de la misma manera que el ejercicio 3, utilizando BFS desde u para obtener el conjunto de vértices alcanzables. El costo computacional en el peor caso es de O(V+E).

Ejercicio 9

Dado un grafo G(A,V) conexo, indique el orden en el cual se pueden ir eliminando los nodos y sus aristas incidentes de manera tal que el grafo que va quedando siempre sea conexo

Se puede resolver utilizando BFS para construir un árbol generador. Luego, se eliminan los vértices que son hojas de dicho árbol, junto con sus aristas incidentes. Se repite el procedimiento hasta que quede un único vértice.

La eliminación de una hoja no desconecta los vértices restantes, ya que el árbol que queda sigue conectándolos a todos. Por lo tanto, el grafo original, después de cada eliminación, continúa siendo conexo mientras queden vértices.

El costo computacional es de O(V+E), correspondiente a la construcción del árbol y a la obtención del orden de eliminación.

Ejercicio 10

Dado un grafo G(A,V) no dirigido y ponderado, determine si existe un camino entre dos nodos u y v cuyo costo total sea menor o igual que un valor K

Se podría usar Dijkstra comenzando desde u, fijándonos cuál es el camino de menor costo hasta v, y si ese valor es menor o igual que K, entonces sí hay, sino no. Esto nos lo garantiza la propiedad de que

cualquier camino entre u y v tendrá un costo mayor o igual que la distancia más corta δ(u,v). El algoritmo corta una vez que se relaja el vértice v

Peor caso y caso promedio: O(E log V). Las operaciones realizadas son de O(log V) y en el peor caso se hará con cada arista.

Ejercicio 11

Dado un grafo G(A,V) dirigido, determine si existe algún nodo desde el cual sea posible alcanzar a todos los demás nodos

Se puede resolver realizando un DFS sobre todo el grafo y registrando el último vértice en finalizar su exploración, es decir, aquel cuyo tiempo de finalización sea máximo. Este vértice se considera un candidato a ser el nodo buscado, ya que, si existe un vértice capaz de alcanzar a todos los demás, el último en finalizar será necesariamente uno de esos posibles candidatos.

Sin embargo, esto no garantiza que el candidato pueda alcanzar a todos los vértices. Por ello, se realiza un segundo recorrido, utilizando BFS desde dicho vértice. Si todos los vértices son visitados, existe un nodo que alcanza a todos los demás; en caso contrario, no existe ninguno.

El costo computacional en el peor caso es de O(V+E), dado que ambos recorridos tienen esa complejidad. En el mejor caso es de O(V), cuando el grafo no posee aristas, debido a la inicialización y el procesamiento de los vértices.

Ejercicio 12

Dado un grafo G(A,V) no dirigido, determine para cada par de nodos si existe más de un camino entre ellos que no comparta ninguna arista

Se puede resolver utilizando DFS para identificar las aristas puente del grafo. Una arista puente es aquella cuya eliminación aumenta la cantidad de componentes conexas.

Si dos vértices pertenecen a la misma componente conexa después de eliminar todas las aristas puente, entonces existe entre ellos más de un camino que no comparte ninguna arista. Si pertenecen a componentes distintas, no existen dos caminos de ese tipo.

Se utiliza esta estrategia porque, en un grafo no dirigido, dos vértices están conectados por al menos dos caminos sin aristas en común si y solo si ninguna arista puente los separa.

El costo computacional para identificar las aristas puente es de O(V+E). Como además se debe determinar la respuesta para cada par de vértices, el costo total es de O(V+E+V^2).

Ejercicio 13

Demuestre que si el grafo G es un árbol y se le agrega una arista nueva entre cualquier par de vértices se forma exactamente un ciclo y deja de ser un árbol.

Para que el grafo G(V,E) sea un árbol debe existir un único camino simple entre cada par de vértices, debe ser conexo, acíclico y tener V-1 aristas.

Sea G=(V,E) un árbol y sean u, v dos vértices arbitrarios de V, tales que (u, v) sea una arista que no pertenece a E. Consideremos el grafo G'=(V, E U {(u, v)})

Dado que G es un árbol, existe un único camino simple p que une u con v, p=(u, x1, ... ,xk, v) con k >=1 donde todos los vértices intermedios son distintos entre sí. En el grafo G' agregando la arista (u, v), la secuencia c quedaría como sigue c=(u,x1,...,xk, v, u). Como p era un camino simple de longitud k>=1, la secuencia c es un ciclo simple en G'. Por lo tanto, se forma al menos un ciclo.

Supongamos que en G' existe un ciclo simple c'. Si c' no contiene a la arista (u, v), entonces todas las aristas de c' pertenecen a E, lo que implicaría que G ya tenía un ciclo antes, lo cual es un absurdo dado que G es un árbol, por lo tanto, todo ciclo en G' debe contener obligatoriamente a (u, v). Si c' contiene a (u, v), entonces al removerlo obtenemos un camino simple p' en G entre u y v, sin embargo, por definición de árbol, existe un único camino entre u y v, así p=p' y c=c'. Así, no pueden existir dos ciclos simples distintos al agregar una arista a un árbol.

Luego, si el grafo G es un árbol y se le agrega una arista nueva entre cualquier par de vértices se forma exactamente un ciclo y deja de ser un árbol.

Ejercicio 14

Demuestre que si la arista (u,v) no pertenece al árbol BFS, entonces los niveles de u y v difieren a lo sumo en 1.

Sea G=(V,E) un grafo conexo, no dirigido y no ponderado, y sea T un árbol BFS obtenido a partir de un vértice inicial s. Denotemos por d(x) el nivel del vértice x, que coincide con la distancia mínima desde s hasta x.

Sean u, v pertenecientes a V dos vértices tales que (u,v) está en E, pero (u,v) no está en T.

Como existe una arista entre u y v, podemos llegar desde s hasta v siguiendo el camino mínimo hasta u y luego recorriendo la arista (u,v). Por lo tanto,

$$d(v) \leq d(u) + 1$$

Análogamente, utilizando el camino mínimo hasta v y luego la arista (v,u), se obtiene:

$$d(u) \leq d(v) + 1$$

Combinando ambas desigualdades, resulta:

$$|d(u) - d(v)| \leq 1$$

Por lo tanto, los niveles de u y v difieren a lo sumo en 1.

Ejercicio 15

Dado un grafo G(A,V) dirigido y ponderado, determine para cada par de nodos u, v u ,v el costo del camino mínimo que conecta u con v, considerando que pueden existir aristas de peso negativo pero no ciclos de peso negativo.

Se puede resolver utilizando el algoritmo de Floyd-Warshall, que calcula las distancias mínimas entre todos los pares de vértices. Se inicializa una matriz D, donde D[i][j] contiene el peso de la arista directa entre i y j, si existe; en caso contrario, se utiliza infinito. Además, D[i][i]=0.

Luego, se consideran sucesivamente todos los vértices k como posibles intermediarios de los caminos entre cada par i, j, actualizando:

$$D[i][j] = min(D[i][j], D[i][k] + D[k][j])$$

Al finalizar, la matriz contiene los costos mínimos entre todos los pares de vértices. El algoritmo admite aristas de peso negativo y, como no existen ciclos de peso negativo, las distancias mínimas están bien definidas entre los vértices alcanzables.

El costo computacional es de O(V^3), debido a los tres ciclos anidados, y el espacio adicional utilizado es de O(V^2) por la matriz de distancias.

Ejercicio 16

Dado un grafo G(A,V) no dirigido y ponderado, determine si existe un camino entre dos nodos uu y vv cuyo costo sea exactamente K.

Si se consideran caminos simples, se puede resolver mediante una búsqueda comenzando desde u y explorando los caminos posibles hacia v. En cada paso se acumula el peso de las aristas recorridas y se evita volver a visitar un vértice que ya pertenece al camino actual.

Si se alcanza v con un costo exactamente igual a K, la respuesta es afirmativa. Si se agotan todos los caminos posibles sin encontrar uno que cumpla la condición, la respuesta es negativa.

En el peor caso, el costo computacional es exponencial, ya que puede ser necesario explorar una cantidad exponencial de caminos simples. Si todos los pesos son positivos, se puede podar una rama cuando el costo acumulado supera K, porque agregar más aristas no permitirá reducirlo.

Ejercicio 17

Demostrar que si la arista (u, v) de costo mínimo tiene un nodo en U y otro en V - U, entonces la arista (u, v) pertenece a un árbol abarcador de costo mínimo.

Sea G=(V,E) un grafo conexo y ponderado, y sea U es un subconjunto de V. Consideremos una arista (u,v), donde u pertenece a U y v pertenece a V-U, cuyo peso es mínimo entre todas las aristas que conectan ambos conjuntos.

Sea T un árbol generador de costo mínimo. Si (u,v) pertenece a T, la propiedad queda demostrada. En caso contrario, agregamos (u,v) a T. Al hacerlo, se forma exactamente un ciclo, porque T es un árbol.

Como el ciclo contiene la arista (u,v), debe existir otra arista (x,y) del ciclo que conecte U con V-U. Por la elección de (u,v), se cumple que:

$$w(u,v) \leq w(x,y)$$

Si eliminamos (x,y) del ciclo, obtenemos nuevamente un árbol generador T', cuyo costo no es mayor que el de T. Como T era un árbol generador de costo mínimo, T' también es un árbol generador de costo mínimo.

Por lo tanto, existe un árbol generador de costo mínimo que contiene a (u,v), como se quería demostrar.

Esta propiedad es la que permite seleccionar con seguridad una arista de peso mínimo que cruza un corte al construir un árbol generador de costo mínimo.

Ejercicio 18

Dado un grafo G(A,V) ponderado y dos nodos uu y vv, determine el camino de menor costo que los conecta. Suponga que todos los pesos de las aristas son positivos.

Se puede resolver utilizando Dijkstra, comenzando desde el vértice u. El algoritmo calcula las distancias mínimas desde u hasta los demás vértices y registra los predecesores necesarios para reconstruir el camino mínimo hasta v.

Al finalizar, se reconstruye el camino siguiendo los predecesores desde v hasta u. Si v no es alcanzable desde u, no existe un camino que los conecte.

Ejercicio 19

Dado un grafo G(A,V) conexo y ponderado, determine el segundo árbol generador de menor costo.

Se puede resolver primero utilizando Kruskal o Prim para obtener un árbol generador de costo mínimo T. Luego, se consideran las aristas que pertenecen al grafo original, pero no a T.

Para cada arista (u,v) que no pertenece a T, se la agrega al árbol, formando un único ciclo. Después, se elimina del ciclo una arista de mayor peso perteneciente a T, obteniendo un nuevo árbol generador. Se calcula el costo de cada uno de estos árboles y se selecciona el de menor costo entre los que son distintos de T.

Esto permite encontrar el segundo árbol generador de menor costo, ya que cualquier árbol generador alternativo puede obtenerse mediante intercambios de aristas respecto de T.

Si se busca el segundo árbol de costo estrictamente mayor que el mínimo, se deben descartar los candidatos cuyo costo sea igual al del AACM original. Si se admite un árbol distinto con el mismo costo mínimo, esos candidatos también se consideran.

Con una implementación sencilla que recorre el árbol para encontrar la arista de mayor peso en cada ciclo, el costo computacional es de O(|E||V|), una vez obtenido el AACM

Ejercicio 20

Dado un grafo G(A,V) conexo y ponderado, determine qué ocurre con el costo del árbol generador mínimo si se incrementa en una unidad el peso de una única arista. Diseñe una estrategia que permita determinar si el árbol generador mínimo cambia.

Si la arista incrementada no pertenece al AACM, no cambia.

Si la arista sí pertenece al AACM, entonces se puede proceder con eliminar la arista del árbol y quedarnos con dos componentes conexas U y V-U. Se verifica cuál es la arista de costo mínimo que une ambas componentes. En el caso de que sea la arista eliminada, entonces el AACM no cambia, pero su costo total incrementa en 1, si, por otro lado, se puede elegir otra arista, entonces sí cambia únicamente reemplazando esas aristas.

El costo computacional adicional es de O(V+E), ya que basta con identificar las dos componentes y recorrer las aristas para encontrar la de menor peso que las conecte

Ejercicio 21

Demuestre que si unimos dos AACM por un arco (arista) de costo mínimo, el resultado es un nuevo AACM. (Base del funcionamiento del algoritmo de Kruskal)

Sean T_1 y T_2 dos árboles generadores de costo mínimo de dos componentes conexas disjuntas, con conjuntos de vértices V_1 y V_2, respectivamente. Sea (u,v) una arista de peso mínimo entre ambas componentes, con u perteneciente a V_1 y v perteneciente a V_2.

Al agregar esta arista a los dos árboles, se obtiene un grafo conexo y acíclico, porque la nueva arista conecta dos componentes que antes estaban separadas. Además, el número total de aristas es:

$$|E(T)| = (|V_1| - 1) + (|V_2| - 1) + 1$$
$$= |V_1| + |V_2| - 1$$
$$= |V| - 1$$

Por lo tanto, el resultado es un árbol generador.

Ahora debemos demostrar que es de costo mínimo. Supongamos que existe otro árbol generador T' de menor costo que no utiliza la arista (u,v). Como T' debe conectar ambas componentes, necesariamente contiene otra arista (x,y) que conecta V_1 con V_2.

Por ser (u,v) una arista de peso mínimo entre ambas componentes, se cumple:

$$w(u, v) \leq w(x, y)$$

Podemos reemplazar (x,y) por (u,v) en T', obteniendo un árbol generador cuyo costo no aumenta. Esto contradice que existiera un árbol generador de costo estrictamente menor que el obtenido al unir T_1 y T_2.

En consecuencia, al unir dos componentes mediante una arista de costo mínimo se obtiene un árbol generador de costo mínimo. Esta propiedad justifica la elección de aristas que realiza Kruskal durante la construcción del AACM.

Ejercicio 22

Explique qué modificaciones habría que hacer en el algoritmo de Prim sobre el grafo no dirigido y conexo G(V,A), o sobre la función de costo c(v1,v2)-> R para lograr:

1. **Obtener un árbol de recubrimiento de costo máximo.**

Se puede modificar el algoritmo de Prim para que, en lugar de seleccionar en cada paso la arista de menor costo que conecta un vértice de U con otro de V-U, seleccione la arista de mayor costo que conecte ambos conjuntos. De esta manera, se obtiene un árbol de recubrimiento de costo máximo, ya que se priorizan las aristas de mayor peso sin formar ciclos.

2. **Obtener un árbol de recubrimiento cualquiera.**

Se puede modificar el algoritmo de Prim para que, en cada paso, seleccione cualquier arista que conecte un vértice de U con otro de V-U, sin considerar su costo. De esta manera, se incorporan vértices al árbol sin formar ciclos hasta que todos los vértices del grafo estén conectados. Así se obtiene un árbol de recubrimiento, aunque no necesariamente de costo mínimo ni máximo.

3. **Dado un conjunto de aristas E ∈ A, que no forman un ciclo, encontrar el árbol de recubrimiento mínimo $G^c(V,A^c)$ tal que E ∈ A^c.**

Se puede modificar el algoritmo de Prim para que las aristas de E se incorporen obligatoriamente al árbol antes de seleccionar las restantes. Como estas aristas no forman ciclos,

pueden agregarse sin perder la propiedad de que el árbol sea acíclico. Luego, se continúa ejecutando Prim, seleccionando en cada paso la arista de menor costo que conecte un vértice de U con otro de V-U, evitando formar ciclos y respetando las aristas ya incorporadas. De esta manera, se obtiene un árbol de recubrimiento mínimo que contiene todas las aristas de E.

Ejercicio 23

Sea **G<V, A>** un grafo conexo, no dirigido y ponderado, donde todas las aristas tienen el mismo costo. Suponiendo que G está implementado usando matriz de adyacencia, haga en pseudocódigo un algoritmo **$O(V^2)$** que devuelva una matriz M de VxV donde: M[u, v] = 1 si (u,v) ∈ A y (u, v) estará obligatoriamente en todo árbol abarcador de costo mínimo de **G**, y cero en caso contrario.

Ejercicio 24

Implementar el Algoritmo de Dijkstra que responde a la siguiente especificación

def shortestPath(Grafo, s, v):

> **Descripción:** Implementa el algoritmo de Dijkstra
> **Entrada: Grafo** con la representación de Matriz de Adyacencia, vértice de inicio **s** y destino **v**.
> **Salida:** retorna la lista de los vértices que conforman el caminoiniciando por **s** y terminando en **v**. Devolver NONE en caso que no exista camino entre **s** y **v**.